

How to learn the talk with this booklet

The booklet has three parts, each with one page per slide. Work through them in order, slide by slide, always **aloud**, standing if you can.

1. **Learn the skeleton, not the words.** Per slide, learn verbatim only the *opening sentence*, the *last sentence* (it is the bridge to the next slide) and the *numbers*. Everything in between is a list of points you say in your own words. A talk recited word for word sounds recited and collapses on the first slip; anchor sentences give you a safe place to land.
2. **Part A – full text.** One sentence per line; every line break is a pause. Read the slide twice, then cover the text and say it while looking at the thumbnail.
3. **Part B – first letters.** Say the slide from the skeleton. Peek only when stuck. Move on once you manage two clean runs.
4. **Part C – anchors only.** Say the whole slide from the opening and closing sentence alone. This is the level you need on stage.
5. **Time it.** Each page carries its time window. The pauses are part of the timing. Target for the whole talk: **12:00**.
6. **Space it out.** Day 1: slides 1–6 to Part C. Day 2: slides 7–13, then re-test 1–6. Day 3: two full runs with the PDF full-screen and a clicker, one of them recorded on your phone – listen back. Day 4: open the booklet on a random page and start from there (this trains recovery from a blank), plus the questions in QA.md. Last evening: one relaxed run. **Freeze the script two days before the talk;** every edit resets what you have learnt.
7. **Anki.** `anki.csv` holds one card per slide, one card per bridge sentence and one per number – for the minutes between sessions.

Legend. **[CLICK]** – advance the slide. *grey italics* – **[CUT IF LATE]**: can be dropped without breaking the argument. Numbers table on the last page.



Good morning.

My name is Martin Maxa, and I'm from the Czech Technical University in Prague, the Faculty of Electrical Engineering.

This is joint work with my colleagues Jakub Svatoš and Jakub Zelinka.

Today I'll show you a small neural network that recognizes the sound of an artillery launch.

It runs on a cheap microcontroller, and it keeps working in noise.

Motivation and research questions

- Passive acoustic sensing supports early warning: it is **cheap**, needs **little power** and **emits no signal** of its own.
- A deployable sensor has to meet two constraints at once:
 - **robust to noise**
background noise,
event variability
 - **runs on an MCU**
low power, little memory, int8

This study

1. Noise robustness through training augmentation: is perturbing the *MFCC features* enough, or do we have to add noise to the *waveform*?
2. Does that robustness survive int8 quantization and deployment on an ESP32-S3?

Post-Trigger Artillery Launch Classifier · BEC 2026

1/12

Let me start with the motivation.

Recent conflicts have shown that we need cheap systems that can warn about artillery fire.

Acoustic sensing is a good candidate.

It is passive, it needs little power, and it doesn't send out any signal, so it is hard to find.

But a real sensor has two problems at the same time.

It has to work in noise.

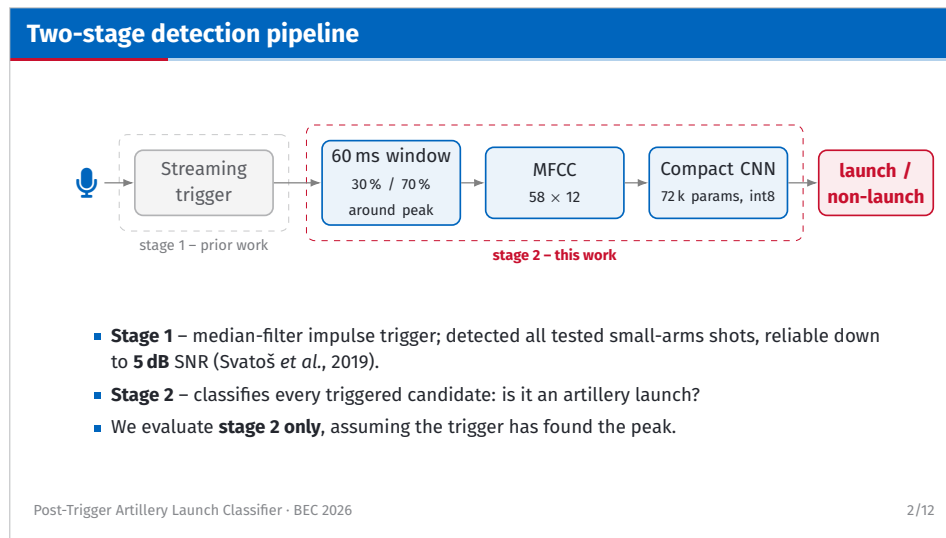
And it has to run on small, low-power hardware.

[CLICK] _____

So we ask two questions.

First – to make the classifier robust to noise, is it enough to add noise to the features, or do we have to add it to the audio signal itself?

And second – does this robustness survive when we quantize the model and run it on a microcontroller?



Our system has two stages.

The first stage is a streaming trigger.

It listens all the time and looks for impulsive events.

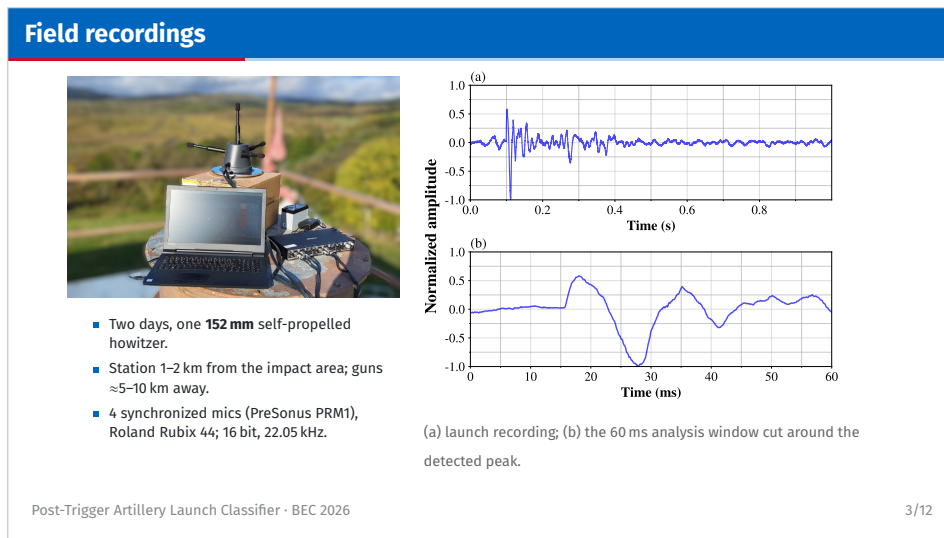
It is our earlier work – it detected all the gunshots we tested, and it works down to a signal-to-noise ratio of five decibels.

The second stage is what this talk is about.

For each candidate, we cut a sixty millisecond window around the peak, compute MFCC features, and a small CNN decides: artillery launch, or not.

So to be clear – we evaluate only the second stage.

We assume the trigger has already found the event.



The data come from real artillery measurements.

This is our recording station.

Four synchronized microphones on one mount, an audio interface and a laptop.

We recorded for two days, with one type of gun – a 152 millimetre self-propelled howitzer.

The station was one to two kilometres from the impact area, and the guns fired from about five to ten kilometres away.

On the right is one launch.

Below it, you can see the sixty millisecond window that the classifier actually sees.

Thirty percent of it is before the peak, and seventy percent after it.

Labelled corpus and event-level split				
Subset	Events	Samples		
		Train	Val.	Test
Artillery launch	50	128	8	10
Impulsive noise	706	457	109	140
Small-arms shots	85	49	18	18
Total	841	634	135	168

- Non-launch: false-trigger candidates – door impacts, shouts, bells, **small-arms gunshots** as hard negatives.
- Split **by event**: the four channels of one shot never cross partitions.
- Training uses all 4 channels (launch share 20 %, class-weighted loss); validation and test use one.

Post-Trigger Artillery Launch Classifier · BEC 2026 4/12

This is the whole labelled corpus – eight hundred and forty-one events.

Fifty of them are artillery launches.

The rest are things that can also fire the trigger – doors, shouts, bells, and small-arms gunshots.

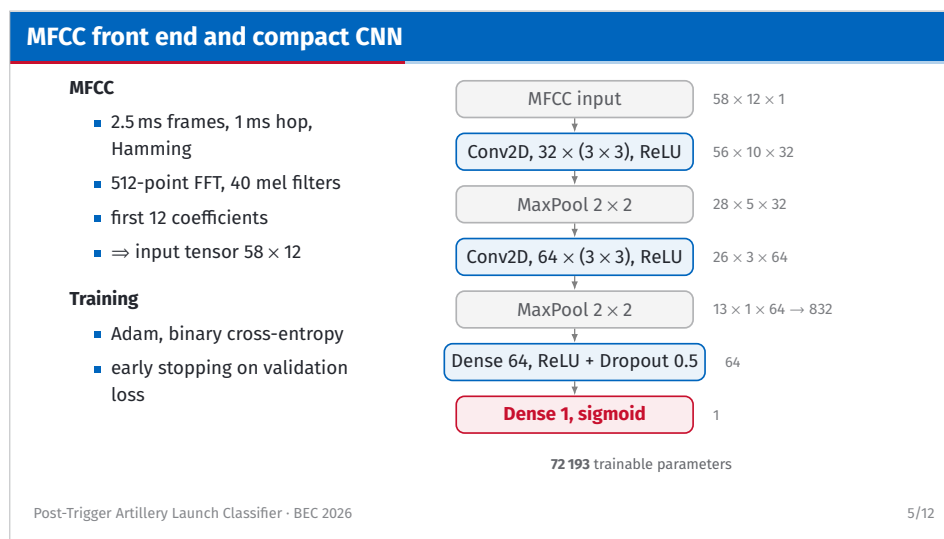
The gunshots are the hard cases, because they are impulsive too.

We split the data by event, not by recording.

So the four channels of one shot are never in training and test at the same time.

In training we use all four channels, and a class-weighted loss for the imbalance.

The test set has ten launches and one hundred and fifty-eight other events.



Now the method.

From each window we compute MFCCs.

We use very short frames – two and a half milliseconds, with a one millisecond step – because the event is short and changes fast.

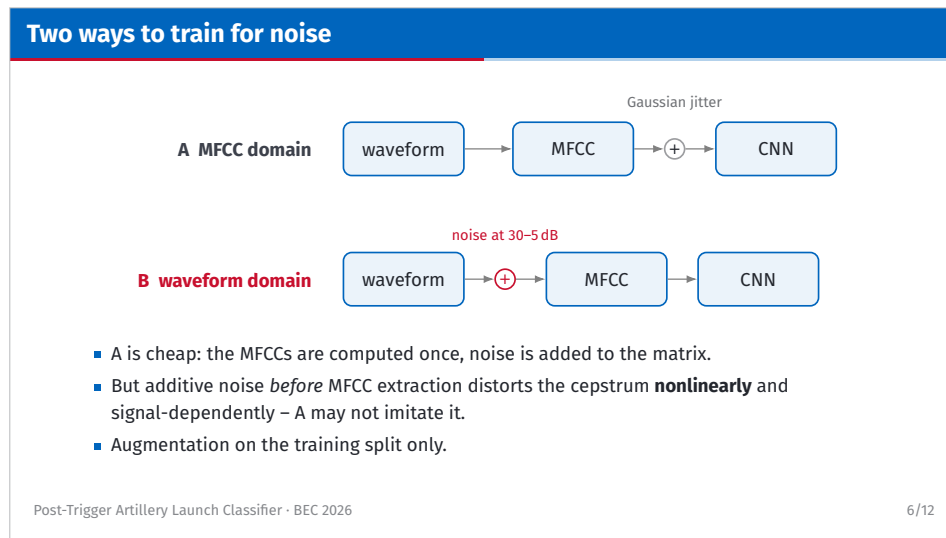
We keep twelve coefficients, so each event becomes a small matrix, fifty-eight by twelve.

The classifier is a small CNN.

Two convolution blocks, each with max pooling, then one dense layer with dropout, and a sigmoid output.

In total it has about seventy-two thousand parameters.

That is small enough for a microcontroller.



To make the network robust, we add noise during training.

And there are two places where we can add it.

Option A: we compute the MFCCs once, and then add small random noise directly to the feature matrix.

This is cheap and simple.

Option B: we add noise to the audio signal first, and only then compute the MFCCs.

This costs more, because we have to recompute the features every time.

The question is whether A is good enough.

Noise in the audio does not stay simple after the MFCC – it changes the features in a nonlinear way.

So A may not look like real noise at all.

Evaluation protocol

- Held-out test partition, used only for the final numbers.
- Noise added to the **waveform before MFCC extraction**, per event:

$$\sigma_n^2 = P_x \cdot 10^{-\text{SNR}/10}$$

at **30, 20, 10, 5 dB** – plus 0 and –5 dB as a stress test, beyond the trigger's 5 dB envelope.

- Peak detection and windowing re-run on the noisy signal.
- **5 training seeds** × **5 noise seeds** = 25 runs per cell, mean ± std; threshold 0.5.
- Three regimes: **float32** and **int8** on the PC, int8 on the **ESP32-S3**.
- Main metric: Matthews correlation coefficient (MCC).

Post-Trigger Artillery Launch Classifier · BEC 2026

7/12

How do we test it?

We take the held-out test set and add noise to the audio – again before the MFCC, because that's what happens in the real sensor.

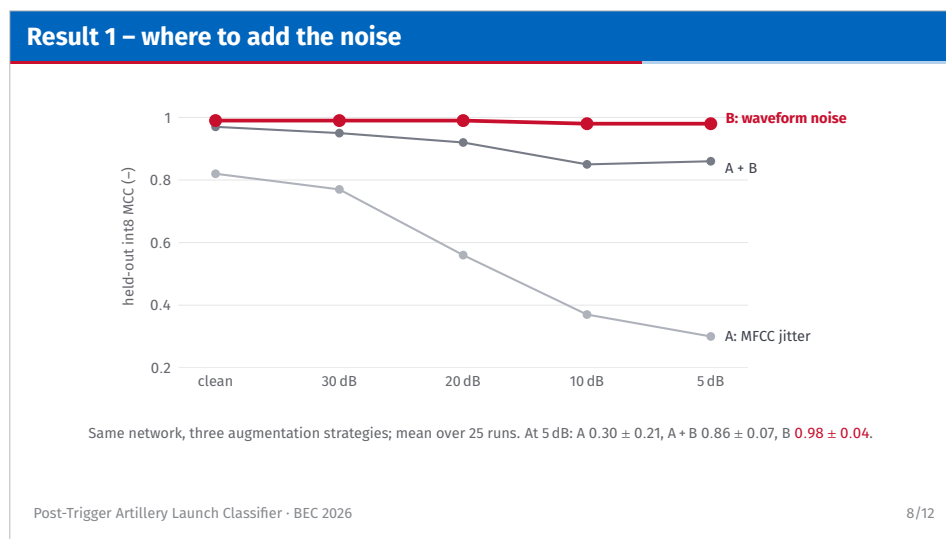
The noise level is set per event, from the power of the event itself.

We test thirty, twenty, ten and five decibels.

And then zero and minus five, which is already beyond what the trigger can handle.

Every number is an average over twenty-five runs – five training seeds and five noise seeds.

And we report mainly the MCC, because the classes are imbalanced.



This is the first result.

Same network, trained three times – only the augmentation is different.

The light grey line is option A, noise on the features.

It is fine on clean data, but it falls apart in noise.

At five decibels the MCC is only zero point three.

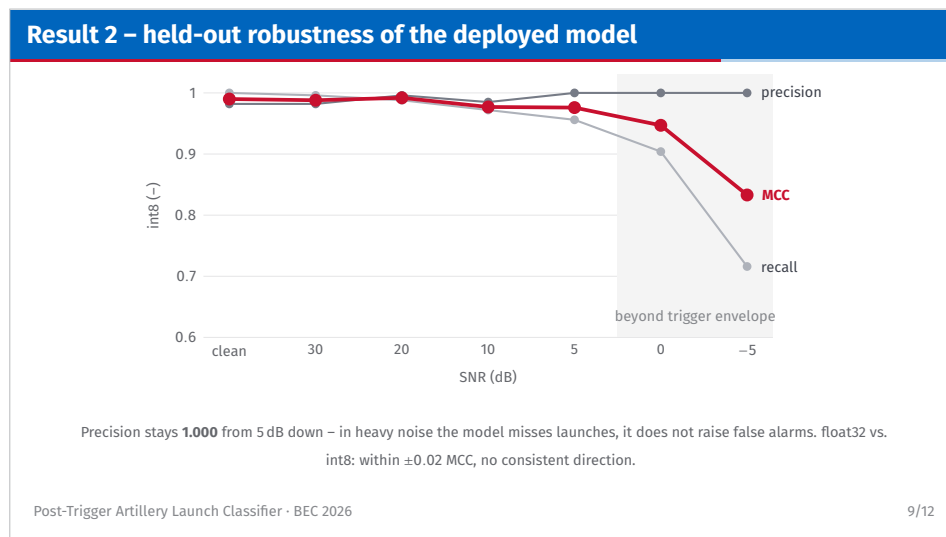
The red line is option B, noise on the waveform.

It stays almost flat – zero point nine eight even at five decibels.

And the surprise is the middle line.

When we combine both, it is worse than B alone.

So feature noise is not just insufficient – here it actually hurts.



Now the deployed model in more detail.

In the trained range, from clean down to five decibels, the MCC stays between zero point nine eight and zero point nine nine.

In the grey area, beyond what the trigger can handle, it goes down slowly. Zero point nine five at zero decibels, and zero point eight three at minus five. But look at how it fails.

Precision stays at one.

Only recall drops.

So in very strong noise the model misses some launches – but it does not give false alarms.

And no small-arms gunshot was ever classified as artillery.

Quantization to int8 made no real difference.

Result 3 – on the ESP32-S3

Same answers as the PC

1171 of 1176 inferences bit-exact with PC int8.
The 5 others: scores at exactly 0.5 – a legal one-bit rounding difference between TFLite and TFLite Micro kernels.

Latency

32.0 ms per MFCC segment (31.99–32.09 ms), independent of noise.
Classifier inference only – MFCC computed on the host.

TensorFlow Lite Micro, int8

Model size	81 400 B
tensor_arena	80 KiB
Flash – code	201.5 kB
Flash – data	144.0 kB
RAM	143.8 kB

All 8 operators native in TFLite Micro; compute-heavy ones via ESP-NN kernels.

Post-Trigger Artillery Launch Classifier · BEC 2026

10/12

Finally, the microcontroller.

We sent all test inputs to the ESP32-S3, at all noise levels – that’s one thousand one hundred and seventy-six inferences.

One thousand one hundred and seventy-one gave exactly the same result as the PC.

The other five had a score of exactly zero point five, right on the decision boundary, where a one-bit rounding difference can flip the answer.

One inference takes thirty-two milliseconds.

Please note, that’s only the network – the MFCCs were computed on the host.

The model has about eighty kilobytes, and the whole firmware needs less than one hundred and fifty kilobytes of RAM.

Limitations

- **Stage 2 only** – no continuous-stream false-trigger rate; results assume the trigger found the peak.
- **Small positive class** – 50 launch events, one gun type (152 mm); other calibres untested.
- **Synthetic Gaussian noise** – closer to reality than feature jitter, but no field noise, propagation or device shift.
- **One event-level split**; no session-independent evaluation.
- **MFCC on the host** – on-device MFCC, end-to-end latency and energy per decision are future work.

Post-Trigger Artillery Launch Classifier · BEC 2026

11/12

Of course, there are limitations.

We evaluate only the classifier, not the full stream.

We have only fifty launches, and one type of gun.

And the noise is synthetic.

So these are the next steps for us.

Conclusion

- A compact MFCC-CNN (72 k parameters) works as the post-trigger stage of an embedded artillery launch detector.
- **Add the noise to the waveform, not to the features:** MFCC-domain jitter collapses at 5 dB (MCC 0.30) and even hurts when combined.
- Waveform-trained model: **MCC 0.99 at 30 dB, 0.98 at 5 dB**; precision 1.0 under heavy noise.
- int8 is performance-neutral; ESP32-S3 matches the PC, 32 ms per inference.

Thank you – questions?

maxamart@fel.cvut.cz

Post-Trigger Artillery Launch Classifier · BEC 2026

12/12

To sum up.

A small CNN with seventy-two thousand parameters works well as the second stage of an artillery launch detector.

If you want it robust to noise, add the noise to the audio, not to the features. Feature noise fails, and combined with waveform noise it even makes things worse.

Our model keeps an MCC of zero point nine eight at five decibels, and it runs on an ESP32-S3 in thirty-two milliseconds.

Thank you for your attention.

I'm happy to take your questions.



G m.

M n i M M, a I'm f t C T U i P, t F o E E.

T i j w w m c J S a J Z.

T I'll s y a s n n t r t s o a a l.

I r o a c m, a i k w i n.

Motivation and research questions

- Passive acoustic sensing supports early warning: it is **cheap**, needs **little power** and **emits no signal** of its own.
- A deployable sensor has to meet two constraints at once:
 - 
robust to noise
background noise,
event variability
 - 
runs on an MCU
low power, little memory, int8

This study

- Noise robustness through training augmentation: is perturbing the *MFCC features* enough, or do we have to add noise to the *waveform*?
- Does that robustness survive int8 quantization and deployment on an ESP32-S3?

Post-Trigger Artillery Launch Classifier · BEC 2026

1/12

L m s w t m.

R c h s t w n c s t c w a a f.

A s i a g c.

I i p, i n l p, a i doesn't s o a s, s i i h t f.

B a r s h t p a t s t.

I h t w i n.

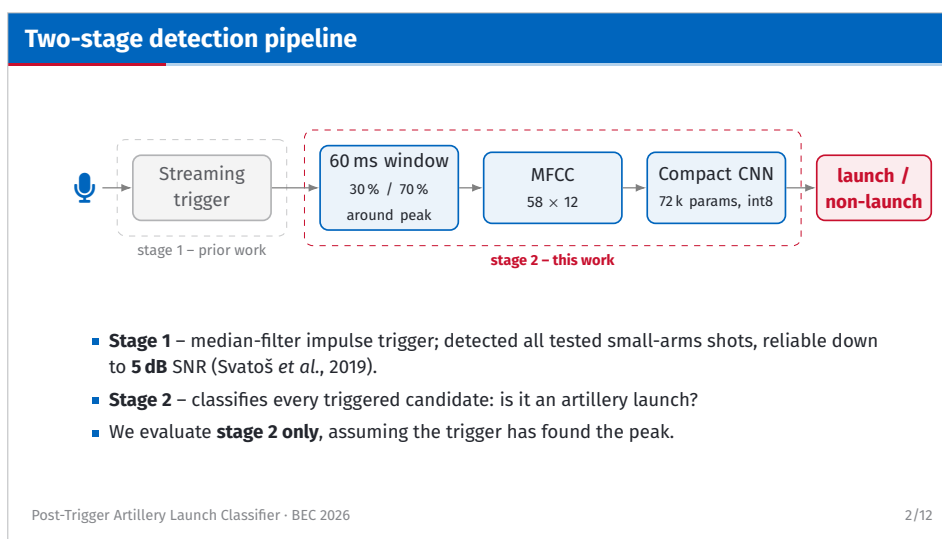
A i h t r o s, l-p h.

[CLICK] _____

S w a t q.

F – t m t c r t n, i i e t a n t t f, o d w h t a i t t a s i?

A s – d t r s w w q t m a r i o a m?



O s h t s.

T f s i a s t.

I l a t t a l f i e.

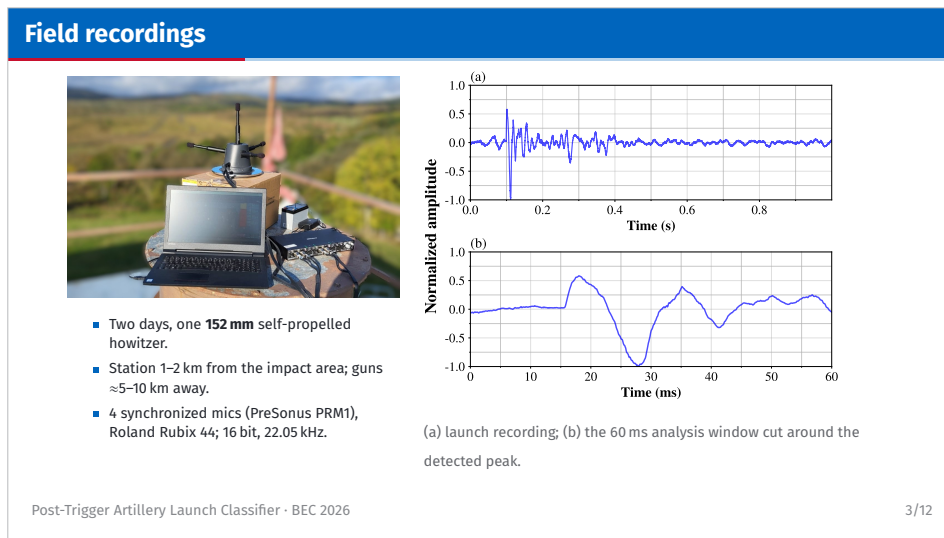
I i o e w – i d a t g w t, a i w d t a s - t - n r o f d.

T s s i w t t i a.

F e c, w c a s m w a t p, c M F C C f, a a s C N N d: a l, o n.

S t b c – w e o t s s.

W a t t h a f t e.



T d c f r a m.

T i o r s.

F s m o o m, a a i a a l.

W r f t d, w o t o g – a 152 m s-p h.

T s w o t t k f t i a, a t g f f a f t t k a.

O t r i o l.

B i, y c s t s m w t t c a s.

T p o i i b t p, a s p a i.

Labelled corpus and event-level split

Subset	Events	Samples		
		Train	Val.	Test
Artillery launch	50	128	8	10
Impulsive noise	706	457	109	140
Small-arms shots	85	49	18	18
Total	841	634	135	168

- Non-launch: false-trigger candidates – door impacts, shouts, bells, **small-arms gunshots** as hard negatives.
- Split **by event**: the four channels of one shot never cross partitions.
- Training uses all 4 channels (launch share 20 %, class-weighted loss); validation and test use one.

Post-Trigger Artillery Launch Classifier · BEC 2026

4/12

T i t w l c – e h a f-o e.

F o t a a l.

T r a t t c a f t t – d, s, b, a s-a g.

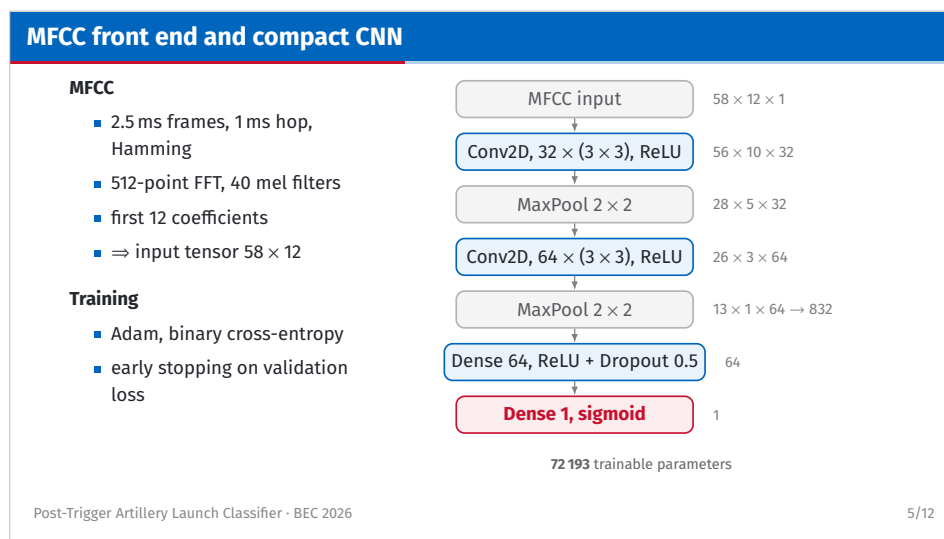
T g a t h c, b t a i t.

W s t d b e, n b r.

S t f c o o s a n i t a t a t s t.

I t w u a f c, a a c-w l f t i.

T t s h t l a o h a f-e o e.



N t m.

F e w w c M.

W u v s f – t a a h m, w a o m s – b t e i s a c f.

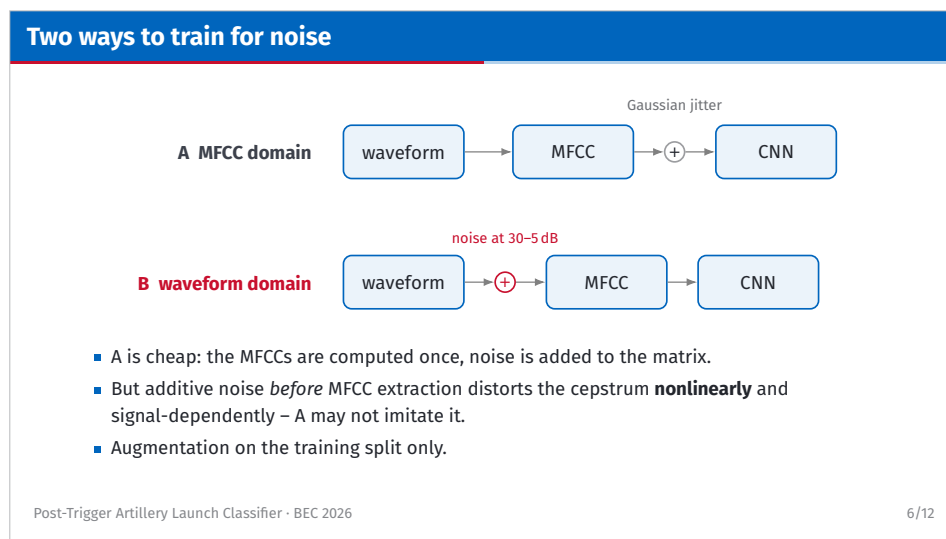
W k t c, s e e b a s m, f-e b t.

T c i a s CNN.

T c b, e w m p, t o d l w d, a a s o.

I t i h a s-t t p.

T i s e f a m.



T m t n r, w a n d t.

A t a t p w w c a i.

O A: w c t M o, a t a s r n d t t f m.

T i c a s.

O B: w a n t t a s f, a o t c t M.

T c m, b w h t r t f e t.

T q i w A i g e.

N i t a d n s s a t M F C C – i c t f i a n w.

S A m n l l r n a a.

Evaluation protocol

- Held-out test partition, used only for the final numbers.
- Noise added to the **waveform before MFCC extraction**, per event:

$$\sigma_n^2 = P_x \cdot 10^{-\text{SNR}/10}$$

at **30, 20, 10, 5 dB** – plus 0 and –5 dB as a stress test, beyond the trigger's 5 dB envelope.

- Peak detection and windowing re-run on the noisy signal.
- **5 training seeds** × **5 noise seeds** = 25 runs per cell, mean ± std; threshold 0.5.
- Three regimes: **float32** and **int8** on the PC, int8 on the **ESP32-S3**.
- Main metric: Matthews correlation coefficient (MCC).

Post-Trigger Artillery Launch Classifier · BEC 2026

7/12

H d w t i?

W t t h-o t s a a n t t a – a b t MFCC, b that's w h i t r s.

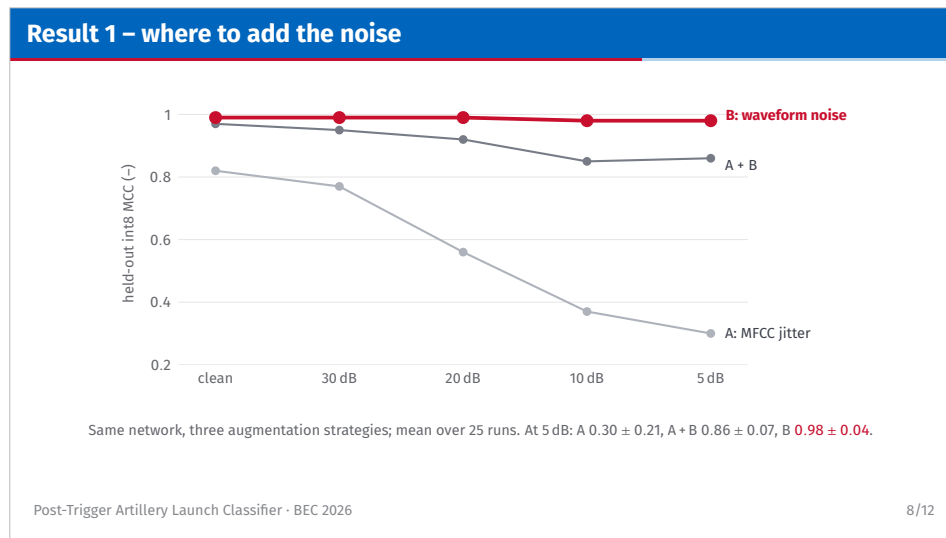
T n l i s p e, f t p o t e i.

W t t, t, t a f d.

A t z a m f, w i a b w t t c h.

E n i a a o t-f r – f t s a f n s.

A w r m t MCC, b t c a i.



T i t f r.

S n, t t t – o t a i d.

T l g l i o A, n o t f.

I i f o c d, b i f a i n.

A f d t MCC i o z p t.

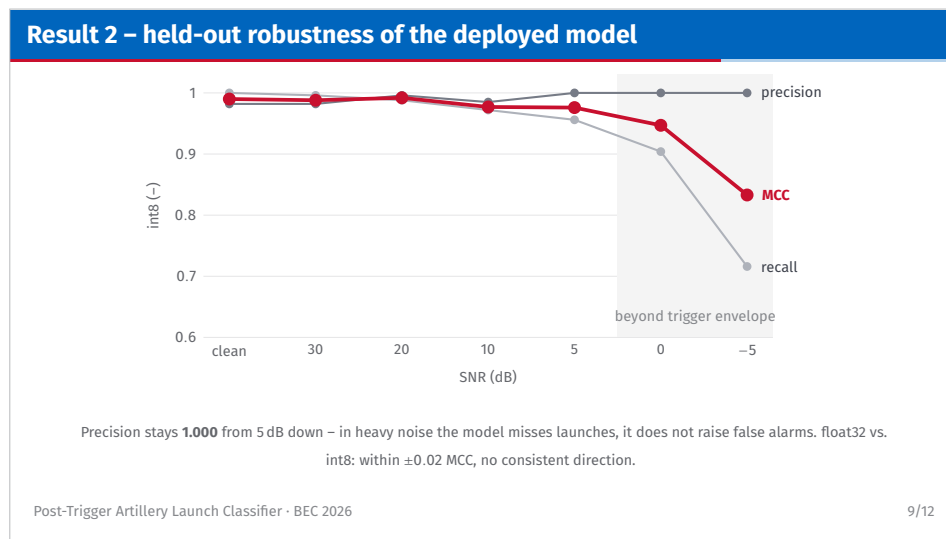
T r l i o B, n o t w.

I s a f – z p n e e a f d.

A t s i t m l.

W w c b, i i w t B a.

S f n i n j i – h i a h.



N t d m i m d.

I t t r, f c d t f d, t MCC s b z p n e a z p n n.

I t g a, b w t t c h, i g d s.

Z p n f a z d, a z p e t a m f.

B l a h i f.

P s a o.

O r d.

S i v s n t m m s l – b i d n g f a.

A n s-a g w e c a a.

Q t int8 m n r d.

Result 3 – on the ESP32-S3

Same answers as the PC

1171 of 1176 inferences bit-exact with PC int8.
The 5 others: scores at exactly 0.5 – a legal one-bit rounding difference between TFLite and TFLite Micro kernels.

Latency

32.0 ms per MFCC segment (31.99–32.09 ms), independent of noise.
Classifier inference only – MFCC computed on the host.

TensorFlow Lite Micro, int8	
Model size	81 400 B
tensor_arena	80 KiB
Flash – code	201.5 kB
Flash – data	144.0 kB
RAM	143.8 kB

All 8 operators native in TFLite Micro; compute-heavy ones via ESP-NN kernels.

Post-Trigger Artillery Launch Classifier · BEC 2026

10/12

F, t m.

W s a t i t t ESP32-S3, a a n l – that's o t o h a s-s i.

O t o h a s-o g e t s r a t PC.

T o f h a s o e z p f, r o t d b, w a o-b r d c f t a.

O i t t-t m.

P n, that's o t n – t M w c o t h.

T m h a e k, a t w f n l t o h a f k o RAM.

Limitations

- **Stage 2 only** – no continuous-stream false-trigger rate; results assume the trigger found the peak.
- **Small positive class** – 50 launch events, one gun type (152 mm); other calibres untested.
- **Synthetic Gaussian noise** – closer to reality than feature jitter, but no field noise, propagation or device shift.
- **One event-level split**; no session-independent evaluation.
- **MFCC on the host** – on-device MFCC, end-to-end latency and energy per decision are future work.

Post-Trigger Artillery Launch Classifier · BEC 2026

11/12

O c, t a l.

W e o t c, n t f s.

W h o f l, a o t o g.

A t n i s.

S t a t n s f u.

Conclusion

- A compact MFCC-CNN (72 k parameters) works as the post-trigger stage of an embedded artillery launch detector.
- **Add the noise to the waveform, not to the features:** MFCC-domain jitter collapses at 5 dB (MCC 0.30) and even hurts when combined.
- Waveform-trained model: **MCC 0.99 at 30 dB, 0.98 at 5 dB**; precision 1.0 under heavy noise.
- int8 is performance-neutral; ESP32-S3 matches the PC, 32 ms per inference.

Thank you – questions?

maxamart@fel.cvut.cz

Post-Trigger Artillery Launch Classifier · BEC 2026

12/12

T s u.

A s CNN w s-t t p w w a t s s o a a l d.

I y w i r t n, a t n t t a, n t t f.

F n f, a c w w n i e m t w.

O m k a MCC o z p n e a f d, a i r o a ESP32-S3 i t-t m.

T y f y a.

I'm h t t y q.



Good morning.

...

It runs on a cheap microcontroller, and it keeps working in noise.

Motivation and research questions

- Passive acoustic sensing supports early warning: it is **cheap**, needs **little power** and **emits no signal** of its own.
- A deployable sensor has to meet two constraints at once:
 - **robust to noise**
background noise,
event variability
 - **runs on an MCU**
low power, little memory, int8

This study

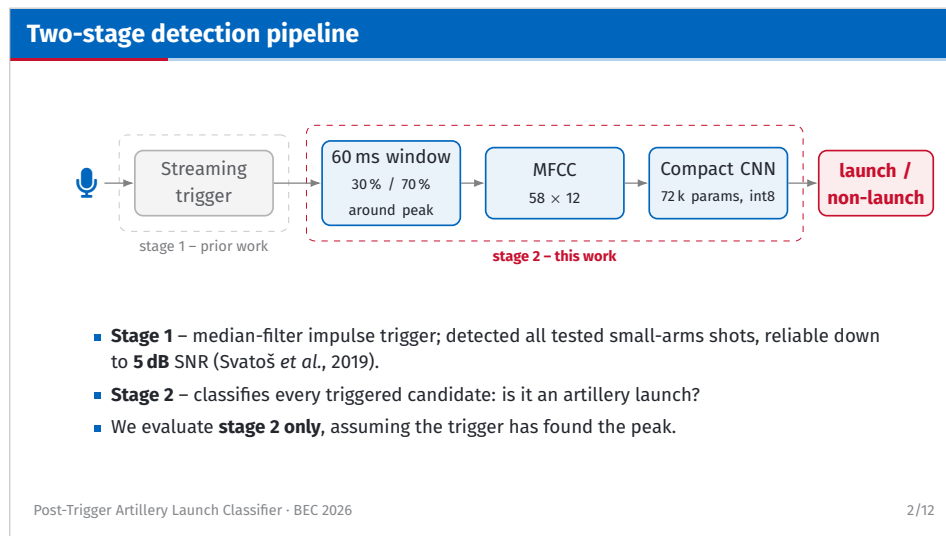
1. Noise robustness through training augmentation: is perturbing the *MFCC features* enough, or do we have to add noise to the *waveform*?
2. Does that robustness survive int8 quantization and deployment on an ESP32-S3?

Post-Trigger Artillery Launch Classifier · BEC 2026 1/12

Let me start with the motivation.

...

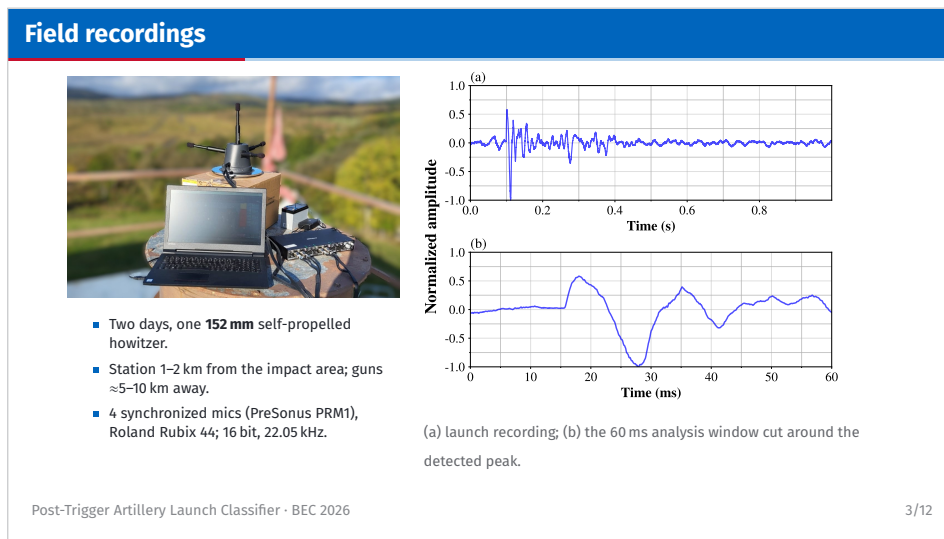
And second – does this robustness survive when we quantize the model and run it on a microcontroller?



Our system has two stages.

...

We assume the trigger has already found the event.



The data come from real artillery measurements.

• • •

Thirty percent of it is before the peak, and seventy percent after it.

Labelled corpus and event-level split				
Subset	Events	Samples		
		Train	Val.	Test
Artillery launch	50	128	8	10
Impulsive noise	706	457	109	140
Small-arms shots	85	49	18	18
Total	841	634	135	168

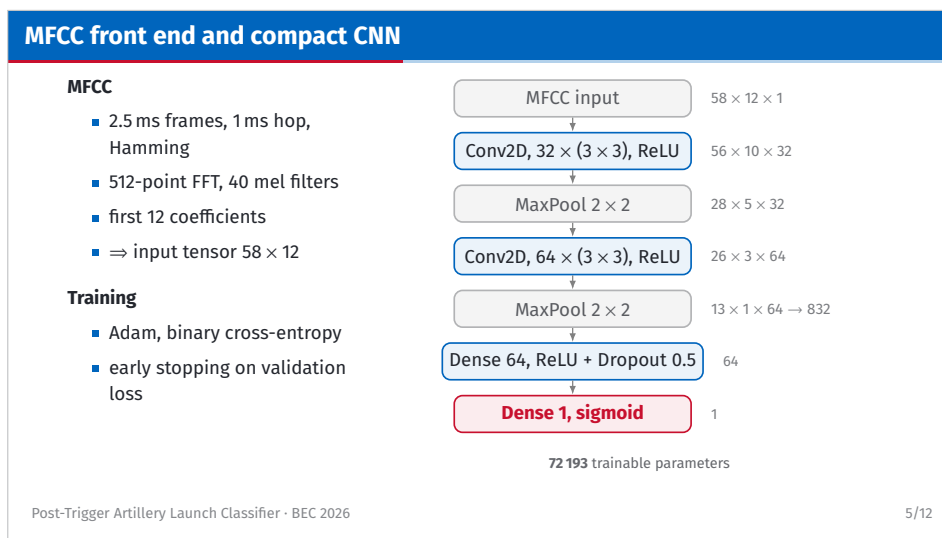
- Non-launch: false-trigger candidates – door impacts, shouts, bells, **small-arms gunshots** as hard negatives.
- Split **by event**: the four channels of one shot never cross partitions.
- Training uses all 4 channels (launch share 20 %, class-weighted loss); validation and test use one.

Post-Trigger Artillery Launch Classifier · BEC 2026 4/12

This is the whole labelled corpus – eight hundred and forty-one events.

• • •

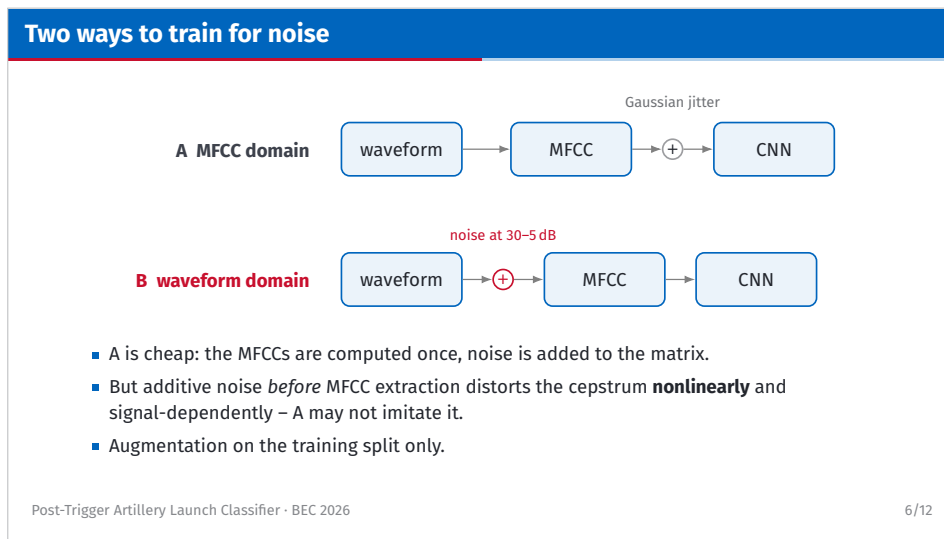
The test set has ten launches and one hundred and fifty-eight other events.



Now the method.

...

That is small enough for a microcontroller.



To make the network robust, we add noise during training.

...

So A may not look like real noise at all.

Evaluation protocol

- Held-out test partition, used only for the final numbers.
- Noise added to the **waveform before MFCC extraction**, per event:

$$\sigma_n^2 = P_x \cdot 10^{-\text{SNR}/10}$$

at **30, 20, 10, 5 dB** – plus 0 and –5 dB as a stress test, beyond the trigger's 5 dB envelope.

- Peak detection and windowing re-run on the noisy signal.
- **5 training seeds** × **5 noise seeds** = 25 runs per cell, mean ± std; threshold 0.5.
- Three regimes: **float32** and **int8** on the PC, int8 on the **ESP32-S3**.
- Main metric: Matthews correlation coefficient (MCC).

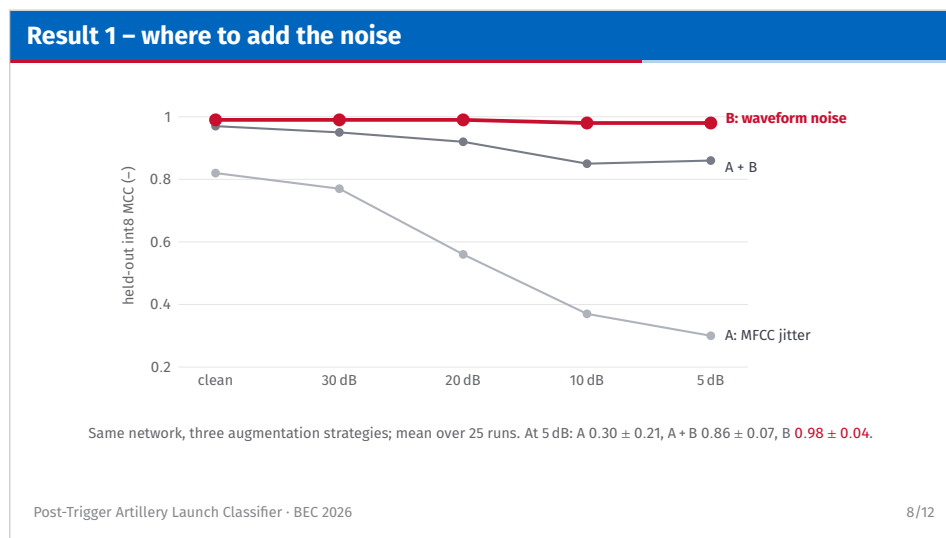
Post-Trigger Artillery Launch Classifier · BEC 2026

7/12

How do we test it?

• • •

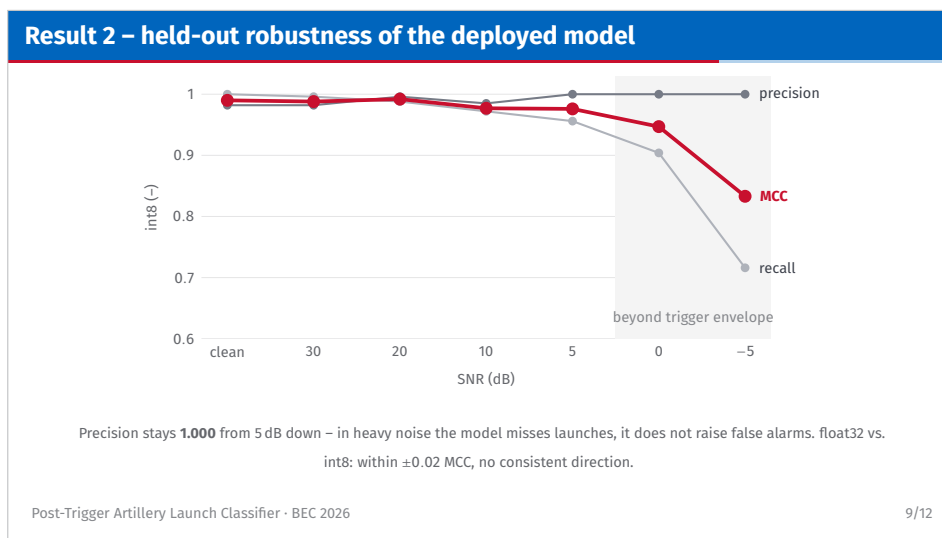
And we report mainly the MCC, because the classes are imbalanced.



This is the first result.

...

So feature noise is not just insufficient – here it actually hurts.



Now the deployed model in more detail.

...

Quantization to int8 made no real difference.

Result 3 – on the ESP32-S3

Same answers as the PC

1171 of 1176 inferences bit-exact with PC int8.
The 5 others: scores at exactly 0.5 – a legal one-bit rounding difference between TFLite and TFLite Micro kernels.

Latency

32.0 ms per MFCC segment (31.99–32.09 ms), independent of noise.
Classifier inference only – MFCC computed on the host.

TensorFlow Lite Micro, int8	
Model size	81400 B
tensor_arena	80 KiB
Flash – code	201.5 kB
Flash – data	144.0 kB
RAM	143.8 kB

All 8 operators native in TFLite Micro; compute-heavy ones via ESP-NN kernels.

Post-Trigger Artillery Launch Classifier · BEC 2026

10/12

Finally, the microcontroller.

...

The model has about eighty kilobytes, and the whole firmware needs less than one hundred and fifty kilobytes of RAM.

Limitations

- **Stage 2 only** – no continuous-stream false-trigger rate; results assume the trigger found the peak.
- **Small positive class** – 50 launch events, one gun type (152 mm); other calibres untested.
- **Synthetic Gaussian noise** – closer to reality than feature jitter, but no field noise, propagation or device shift.
- **One event-level split**; no session-independent evaluation.
- **MFCC on the host** – on-device MFCC, end-to-end latency and energy per decision are future work.

Post-Trigger Artillery Launch Classifier · BEC 2026

11/12

Of course, there are limitations.

• • •

So these are the next steps for us.

Conclusion

- A compact MFCC-CNN (72 k parameters) works as the post-trigger stage of an embedded artillery launch detector.
- **Add the noise to the waveform, not to the features:** MFCC-domain jitter collapses at 5 dB (MCC 0.30) and even hurts when combined.
- Waveform-trained model: **MCC 0.99 at 30 dB, 0.98 at 5 dB**; precision 1.0 under heavy noise.
- int8 is performance-neutral; ESP32-S3 matches the PC, 32 ms per inference.

Thank you – questions?

maxamart@fel.cvut.cz

Post-Trigger Artillery Launch Classifier · BEC 2026

12/12

To sum up.

• • •

I'm happy to take your questions.

Every number in the talk

Corpus – how many labelled events, how many launches?	841 events, 50 artillery launches
Non-launch events – impulsive noise / small-arms shots?	706 / 85
Test partition – samples, launches?	168 samples, 10 launches (158 non-launch)
Microphones per event, gun type?	4 synchronized mics; one 152 mm self-propelled howitzer
Station distances?	1–2 km from the impact area; guns $\approx$ 5–10 km away
Sampling rate, bit depth?	22.05 kHz, 16 bit
Analysis window?	60 ms, 30 % before the peak / 70 % after
MFCC frames?	2.5 ms frames, 1 ms hop, 512-point FFT, 40 mel filters, 12 coefficients
CNN input tensor?	$58 \times 12 \times 1$
CNN trainable parameters?	72,193
Trigger stage – validated down to which SNR?	5 dB
SNR levels tested?	30, 20, 10, 5 dB + stress 0 and –5 dB
Runs per table cell?	25 = 5 training seeds $\times$ 5 noise seeds
MFCC jitter – int8 MCC at 5 dB?	0.30 ± 0.21
Jitter + waveform noise – int8 MCC at 5 dB?	0.86 ± 0.07
Waveform noise only – int8 MCC at 5 dB?	0.98 ± 0.04
Deployed model – MCC at 30 dB / 5 dB?	0.99 / 0.98 (0.988 / 0.976)
Deployed model – MCC at 0 dB / –5 dB?	0.947 / 0.833
Precision from 5 dB downward?	1.000 – misses launches, no false alarms
float32 vs int8 difference?	within ± 0.02 MCC, no consistent direction
ESP32-S3 – bit-exact inferences?	1,171 of 1,176 (5 at score exactly 0.5)
ESP32-S3 – latency per MFCC segment?	32.0 ms (31.99–32.09 ms), classifier only
Model size, tensor arena?	81,400 B; 80 KiB
Firmware flash code / flash data / RAM?	201.5 kB / 144.0 kB / 143.8 kB
Talk length – target?	script written to 12:00 (slot length to confirm)
